



UNIDAD II · SEGURIDAD

Seguridad con Spring Security y JWT

Continuación del proyecto appweb-api: de un CRUD abierto a una API REST protegida con autenticación, roles y control de acceso.

Spring Boot 4.1 · Spring Security 7.1 · JWT 0.13 · BCrypt · PostgreSQL 18

La ruta: de una API abierta a una protegida



1 • Autenticación vs autorización

Quien eres frente a que puedes hacer.



3 • Dependencias

starter-security y JJWT 0.13.



5 • UserDetailsService

Cargar el usuario desde PostgreSQL.



7 • Filtro y configuración

JwtAuthFilter + SecurityConfig.



9 • Clientes en otro servidor

CORS: Angular vs Thymeleaf.



2 • Como funciona un JWT

Estructura, firma y flujo sin estado.



4 • Usuario, roles y datos

Entidad Usuario, BCrypt y repositorio.



6 • Emitir y validar tokens

JwtService con JJWT.



8 • Registro e inicio de sesion

AuthController y prueba con cURL.



10 • Buenas practicas

Errores comunes y checklist.

Autenticacion y autorizacion: dos preguntas



Autenticacion

Quien eres.

Verifica la identidad comprobando las credenciales (usuario y contraseña). Ocurre una sola vez, al iniciar sesion, y termina con la emision del token.



Autorizacion

Que puedes hacer.

Una vez identificado, decide si tiene permiso para una accion concreta segun su rol. Ocurre en cada peticion posterior.

401 No autenticado = 'no se quien eres'

403 Prohibido = 'se quien eres, pero no puedes'

Una API REST es sin estado (stateless)

El problema

HTTP no recuerda al usuario entre una petición y la siguiente.

El inicio de sesión clásico con cookie de sesión guarda estado en el servidor: no encaja con una API REST distribuida.

La solución: token

Tras autenticarse una vez, el cliente recibe una credencial autocontenida **que envía en cada petición.**

El servidor la valida sin recordar nada: escala en horizontal y encaja con una SPA en Angular.

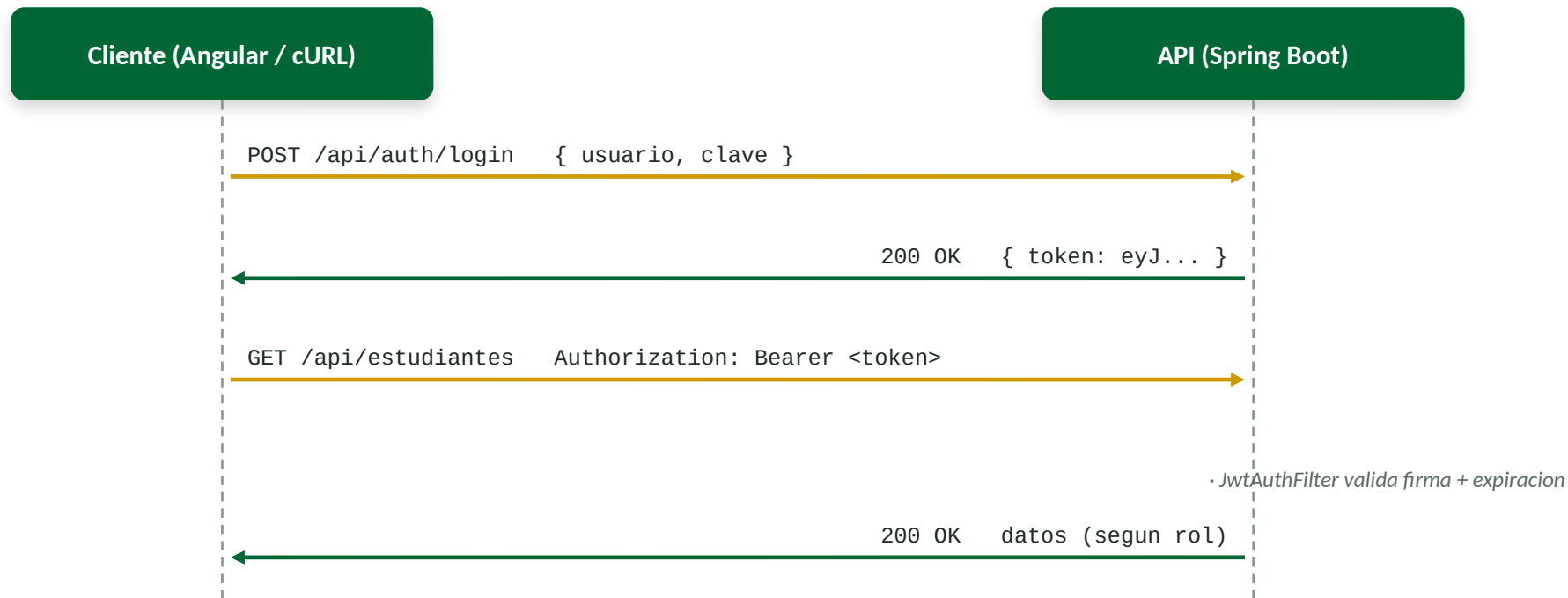
```
alg: HS256
typ: JWT
```

```
sub: usuario
roles: [ADMIN]
exp: 1718...
(legible, no secreto)
```

HMAC-SHA256 con
clave secreta del
servidor

Cabecera y cuerpo van en Base64URL (no cifrados): cualquiera los lee. Lo que protege al token es la firma.

Flujo: iniciar sesion una vez, luego Bearer



La pila verificada (junio 2026)

La seguridad es donde mas cambian las APIs entre versiones. Spring Security 7 elimino metodos antiguos: mucho codigo de tutoriales viejos ya no compila.

Componente	Version	Rol en la seguridad
Spring Boot	4.1.0	Arranque y autoconfiguracion
Spring Security	7.1	Filtros, autenticacion, autorizacion
JWT (io.jsonwebtoken)	0.13.0	Construye y verifica los tokens
Java	25 LTS (o 21 LTS)	Lenguaje y maquina virtual
PostgreSQL	18	Almacena usuarios y roles
BCrypt (Spring Security)	incluido	Cifra y compara contraseñas

pom.xml: security y JWT

pom.xml

```
<!-- Spring Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JSON Web Token (JJWT 0.13.0) -->
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>
<!-- + jjwt-impl y jjwt-jackson (scope runtime) -->
```

application.properties

```
# Secreto de firma (>= 32 bytes para
HS256)
app.jwt.secret=cambia-esta-clave-
larga-aleatoria
# Vida del token en ms (3600000 = 1
hora)
app.jwt.expiration-ms=3600000
```

Aviso

Al añadir el starter, Spring protege TODOS los endpoints por defecto. Los abriremos en el Paso 7.

Usuario y Rol: la identidad en la BD

model/Usuario.java

```
@Entity @Table(name = "usuario")
@Getter @Setter
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false, length = 60)
    private String username;
    @Column(nullable = false)
    private String password;           // hash BCrypt

    @ElementCollection(fetch = FetchType.EAGER)
    @CollectionTable(name = "usuario_rol",
        joinColumns = @JoinColumn(name = "usuario_id"))
    @Enumerated(EnumType.STRING)
    @Column(name = "rol")
    private Set<Rol> roles;
}
```

model/Rol.java

```
public enum Rol {
    ADMIN,
    DOCENTE,
    ESTUDIANTE
}
```

Clave

La contraseña se guarda SIEMPRE cifrada con BCrypt, nunca en texto plano.

Repositorio y tablas que genera Hibernate

```
repository/UsuarioRepository.java
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(String username);
    boolean existsByUsername(String username);
}
```

Dos consultas derivadas

findByUsername lo usa el UserDetailsService.

existsByUsername lo usa el registro.

DDL generado (ddl-auto=update)

```
CREATE TABLE usuario (
  id          BIGINT GENERATED ALWAYS AS IDENTITY,
  username    VARCHAR(60) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  PRIMARY KEY (id)
);
CREATE TABLE usuario_rol (
  usuario_id  BIGINT NOT NULL REFERENCES usuario (id),
  rol         VARCHAR(255)
);
```

Ensenar a Spring a leer tu tabla usuario

```
security/UsuarioDetailsService.java

@Service
@RequiredArgsConstructor
public class UsuarioDetailsService implements UserDetailsService {

    private final UsuarioRepository repo;

    @Override
    public UserDetails loadUserByUsername(String username) {
        Usuario u = repo.findByUsername(username)
            .orElseThrow(() ->
                new UsernameNotFoundException("No existe: " + username));

        var autoridades = u.getRoles().stream()
            .map(r -> new SimpleGrantedAuthority("ROLE_" + r.name()))
            .toList();

        return User.builder().username(u.getUsername())
            .password(u.getPassword()).authorities(autoridades).build();
    }
}
```

Quien cifra y quien autentica

Beans en SecurityConfig

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
public AuthenticationManager authenticationManager(
    UserDetailsService uds, PasswordEncoder encoder) {
    DaoAuthenticationProvider provider =
        new DaoAuthenticationProvider(uds); // constructor SS7
    provider.setPasswordEncoder(encoder);
    return new ProviderManager(provider);
}
```

Patron oficial SS7

El constructor sin argumentos con setUserDetailsService quedo en desuso.

Correcto: new
DaoAuthenticationProvider(uds) +
setPasswordEncoder, envuelto en
ProviderManager.

JwtService: emitir el token

security/JwtService.java – emission

```
@Service
public class JwtService {
    @Value("${app.jwt.secret}")    private String secret;
    @Value("${app.jwt.expiration-ms}") private long expiracionMs;

    private SecretKey clave() {
        return Keys.hmacShaKeyFor(secret.getBytes(UTF_8));
    }

    public String generarToken(UserDetails usuario) {
        Instant ahora = Instant.now();
        return Jwts.builder()
            .subject(usuario.getUsername())
            .claim("roles", usuario.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority).toList())
            .issuedAt(Date.from(ahora))
            .expiration(Date.from(ahora.plusMillis(expiracionMs)))
            .signWith(clave()).compact();
    }
}
```

JwtService: leer y validar el token

security/JwtService.java – validacion

```
public String extraerUsername(String token) {
    return parsear(token).getSubject();
}

public boolean esValido(String token, UserDetails usuario) {
    Claims c = parsear(token);
    return c.getSubject().equals(usuario.getUsername())
        && c.getExpiration().after(new Date());
}

private Claims parsear(String token) {
    return Jwts.parser()
        .verifyWith(clave())           // comprueba la firma
        .build()
        .parseSignedClaims(token)
        .getPayload();
}
```

Que valida

1. **La firma** (verifyWith): nadie altero el token.
2. **El subject** coincide con el usuario.
3. **La expiracion** aun no ha pasado.

JwtAuthFilter: intercepta cada peticion

security/JwtAuthFilter.java (resumen)

@Component

@RequiredArgsConstructor

```
public class JwtAuthFilter extends OncePerRequestFilter {  
    private final JwtService jwtService;  
    private final UserDetailsService userDetailsService;
```

@Override

```
protected void doFilterInternal(HttpServletRequest req,  
    HttpServletResponse res, FilterChain chain) ... {  
    String cabecera = req.getHeader("Authorization");  
    if (cabecera == null || !cabecera.startsWith("Bearer ")) {  
        chain.doFilter(req, res); return; // sin token: sigue  
    }  
    String token = cabecera.substring(7);  
    String username = jwtService.extraerUsername(token);  
    if (username != null && noAutenticadoAun()) {  
        UserDetails u = userDetailsService.loadUserByUsername(username);  
        if (jwtService.esValido(token, u))  
            SecurityContextHolder.getContext().setAuthentication(auth(u));  
    }  
    chain.doFilter(req, res);  
}
```

La cadena de filtros de Spring Security



```
addFilterBefore(jwtAuthFilter,  
UsernamePasswordAuthenticationFilter.class)
```

Registramos el filtro ANTES del de usuario/clave para que el token se procese a tiempo.

SecurityContext: una vez poblado, el controlador conoce al usuario y sus roles.

SecurityConfig: la politica de acceso

```
security/SecurityConfig.java

@Configuration @EnableWebSecurity @EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthFilter jwtAuthFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) ... {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .sessionManagement(sm -> sm
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .addFilterBefore(jwtAuthFilter,
                UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

AuthController: registro e inicio de sesion

```
controller/AuthController.java

@RestController @RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {
    private final AuthenticationManager authManager;
    private final JwtService jwtService;
    private final UsuarioRepository repo;
    private final PasswordEncoder encoder;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest req) {
        Authentication petition = UsernamePasswordAuthenticationToken
            .unauthenticated(req.username(), req.password());
        Authentication auth = authManager.authenticate(petition);
        String token = jwtService.generarToken(
            (UserDetails) auth.getPrincipal());
        return ResponseEntity.ok(new AuthResponse(token, "Bearer", 3600000));
    }
    // + POST /registro: cifra la clave y guarda el Usuario
}
```

@PreAuthorize: reglas finas en el controlador

EstudianteController.java

```
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void eliminar(@PathVariable Long id) {
    servicio.eliminar(id);
}
```

Dos capas que se complementan

Ruta (SecurityConfig): filtra pronto.

Metodo (@PreAuthorize): regla fina sobre un punto concreto.

Defensa en profundidad.

Requisito: @PreAuthorize solo funciona si la clase SecurityConfig lleva **@EnableMethodSecurity** (Paso 7). La expresion puede ser hasRole, hasAnyRole o condiciones mas ricas como #id == authentication.name.

El ciclo completo con cURL

Terminal

```
# 1) login -> token

$ curl -X POST .../api/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"username":"admin","password":"x"}'

< {"token":"eyJ...", "tipo":"Bearer"}

# 2) llamada protegida con el token

$ curl .../api/estudiantes \
  -H 'Authorization: Bearer eyJ...'

< 200 OK    [ ... ]
```

Situación	Respuesta
Credenciales correctas	200 + token
Credenciales incorrectas	401
Sin token (ruta protegida)	401
Token valido, rol insuficiente	403
Token expirado o alterado	401

Swagger con el esquema Bearer

config/OpenApiConfig.java

```
@Configuration
@OpenAPIDefinition(
    info = @Info(title = "API Academica UTEQ", version = "1.0"),
    security = @SecurityRequirement(name = "bearerAuth"))
@SecurityScheme(
    name = "bearerAuth",
    type = SecuritySchemeType.HTTP,
    scheme = "bearer",
    bearerFormat = "JWT")
public class OpenApiConfig { }
```



Boton Authorize

Aparece en Swagger UI; el usuario pega su token y prueba los endpoints protegidos desde el navegador.

/swagger-ui/** y /v3/api-docs/** ya estan permitidos en el Paso 7.

La Semana 6 dejo Swagger funcionando; ahora los endpoints exigen token. Esta anotacion habilita el candado para autenticarse.



CORS: cuando aplica (y cuando no)

CORS es una política del NAVEGADOR: solo interviene cuando la llamada se hace desde un navegador hacia otro origen.

Cliente	Donde corre la llamada	CORS
SPA Angular en otro servidor	Navegador	Si
Thymeleaf (SSR) + RestClient	Servidor (backend)	No
Thymeleaf con fetch/JS a otro origen	Navegador	Si
Postman / cURL / pruebas	Sin navegador	No

Angular: el navegador descarga la SPA y desde ahí llama a la API en otro origen. · Thymeleaf SSR: el servidor llama de backend a backend, sin navegador de por medio.

Configurarlo dentro de Spring Security

SecurityConfig – linea a anadir

```
http
    .cors(Customizer.withDefaults())    // usa el bean de abajo
    .csrf(csrf -> csrf.disable())
    // ... resto de Paso 7 sin cambios ...
```

bean corsConfigurationSource

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(List.of(
        "https://app.uteq.edu.ec", "http://localhost:4200"));
    config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
    config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    config.setAllowCredentials(false);    // token en cabecera, no cookies
    var source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", config);
    return source;
}
```

Dos trampas

El bean debe llamarse exacto corsConfigurationSource, o el preflight se rechaza en silencio.

Nunca '*' junto a allowCredentials(true).

El cliente: Angular y Thymeleaf

Angular – auth.interceptor.ts

```
export const authInterceptor: HttpInterceptorFn =  
  (req, next) => {  
    const token = localStorage.getItem('token');  
    if (token) {  
      req = req.clone({ setHeaders: {  
        Authorization: `Bearer ${token}` } });  
    }  
    return next(req);  
  };
```

Angular adjunta el token automáticamente en cada petición. Aplica CORS (navegador).

Thymeleaf – RestClient (backend)

```
// token guardado en la sesión del usuario  
EstudianteDTO[] datos = restClient.get()  
    .uri(https://api.uteq.edu.ec/api/estudiantes")  
    .header("Authorization", "Bearer " + token)  
    .retrieve()  
    .body(EstudianteDTO[].class);
```

Servidor a servidor: no hay CORS. El reto es custodiar el token.

Buenas practicas y errores comunes



Secreto fuera del codigo

Variable de entorno; ≥ 32 bytes para HS256.



Expiracion corta

1 hora; para sesiones largas, refresh token.



Contraseñas con BCrypt

Con sal y coste; nunca MD5/SHA simple.



Siempre HTTPS

El token viaja en cada peticion.



Nada sensible en el cuerpo

El payload es legible; solo id y roles.



Filtro bien registrado

@Component + addFilterBefore.



C I E R R E

De una API abierta a una API protegida

Añadiste autenticación con **JWT firmado**, contraseñas con BCrypt, un filtro sin estado, control de acceso por rol y CORS para clientes en otro servidor.

Lo que sigue: una SPA en Angular se autenticará y consumirá esta API enviando el token en cada petición.

Gracias